МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»

Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

Лабораторна робота №7

з дисципліни «Алгоритмізація та програмування»

на тему: «Реалізація алгоритмів обробки двовимірних масивів мовою С ++»

ХАІ.301.G12.311.8 ПР

Виконав: здобувач 1 курсу, групи 311
напряму підготовки (спеціальності) G12
Авіаційна та ракетно-космічна техніка

Максим ПАСНІЧЕНКО

Прийняла: доц. каф.301, канд. техн. наук,

_____Олена ГАВРИЛЕНКО

2026

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ представлення двовимірних масивів (матриць) у мові С ++ і реалізувати декларацію, введення з консолі, обробку і виведення в консоль матриць мовою С ++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. Вирішити завдання на аналіз і виведення елементів матриці. Введення і виведення даних здійснити в командному вікні. Варіанти завдань представлено в табл.1.

Завдання 2. Перетворити матрицю відповідно до свого варіанту завдання (див. табл.2.), розмір матриці і його елементи ввести з консолі. Вивести результати у консоль.

Завдання 3. У функції main() організувати багаторазовий вибір одного з двох завдань. Кожне завдання описати окремою функцією без параметрів. Введення, виведення, обробку матриць реалізувати окремими функціями з параметрами.

Завдання 4. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок.

ВИКОНАННЯ РОБОТИ

Завдання 1. Matrix24. Дана матриця розміру M x N. У кожному стовпці матриці знайти максимальний елемент.

Вирішення задачі Matrix24.

Вхідні дані (ім’я, опис, тип, обмеження): m (рядки), п (стовпці) - тип int (від 2 до 50). matr – двовимірний масив int[][] (до 50x50 цілих чисел).

Вихідні дані (ім’я, опис, тип): n цілих чисел (тип int) - максимальні елементи кожного стовпця.

Алгоритм вирішення показано нижче на рис. 1

```
const int M = 50, N = 50; // максимальна кількість рядків і стовпців

// Завдання 1 (Matrix 24): Пошук максимального елемента у кожному стовпці
void print_max_cols(const int matr[M][N], const int m, const int n) {
```

```
cout << "Максимуми по стовпцях:\n";
// Зовнішній цикл по стовпцях (j)
for(int j = 0; j < n; j++) {
int mx = matr[0][j]; // Приймаємо перший елемент стовпця за максимум
// Внутрішній цикл по рядках (i)
for(int i = 1; i < m; i++)
if(matr[i][j] > mx) mx = matr[i][j]; // Оновлюємо максимум
cout << mx << "\t"; // Виводимо знайдений максимум
}
cout << endl;
}
```

Рисунок 1 – Код вирішення задачі Matrix 24 на C ++

Лістинг коду всієї програми наведено в дод. А (стор. 5).

Екран роботи програми Matrix 24 показаний на ст.8 рис. Б1

UML діаграми подано у додатку В

Діалог з ШІ у додатку Г

Завдання 2. Matrix53. Дана матриця розміру M x N. Поміняти місцями стовпець з номером 1 і останній з стовпців, що містять тільки додатні елементи. Якщо необхідних стовпців немає, то вивести матрицю без змін.

Вирішення задачі Matrix53.

Вхідні дані (ім’я, опис, тип, обмеження): (m, n - тип int від 2 до 50; matr масив int[][]).

Вихідні дані (ім’я, опис, тип): matr (тип int[][]) - результуюча матриця після обміну стовпців (або початкова, якщо потрібних стовпців не знайдено).

Алгоритм вирішення показано нижче на рис. 2

```
const int M = 50, N = 50; // максимальна кількість рядків і стовпців

// Завдання 2 (Matrix 53): Перетворення матриці (обмін стовпців)
void change_matr(int matr[M][N], const int m, const int n) {
  // Шукаємо з кінця (справа наліво)
  for(int j = n - 1; j >= 0; j--) {
```

```
            bool pos = true; // Прапорець: чи всі елементи стовпця додатні
            for(int i = 0; i < m; i++) {
                if(matr[i][j] <= 0) { // Якщо знайдено недодатний елемент
                    pos = false;
                    break; // Переходимо до наступного стовпця
                }
            }
            // Якщо знайдено стовпець, де всі елементи додатні
            if(pos) {
                // Міняємо місцями знайдений стовпець (j) і перший стовпець
(0)
                for(int i = 0; i < m; i++) {
                    int temp = matr[i][0];
                    matr[i][0] = matr[i][j];
                    matr[i][j] = temp;
                }
                break; // Робимо заміну тільки для одного (останнього) стовпця
і виходимо
            }
        }
    }
```

Рисунок 2 – Код вирішення задачі Matrix 53 на C ++

ВИСНОВКИ

Під час виконання роботи закріпив практичні навички обробки двовимірних масивів (матриць) та розробки модульних програм мовою C++. Зокрема, було реалізовано алгоритм лінійного пошуку максимальних елементів у кожному стовпці матриці . Також розроблено алгоритм перетворення матриці, який здійснює пошук з кінця, та обмін стовпців за заданою умовою.

ДОДАТОК А
Лістинг коду програми

```
  #include <iostream>

#include <clocale> // Стандартна бібліотека для мовних налаштувань

using namespace std;
const int M = 50,
      N = 50;
// Прототипи функцій
void task1();
void task2();
void get_matr(int in_matr[M][N], int & in_m, int & in_n);
void show_matr(const int out_matr[M][N],
      const int m,
            const int n,
                  const char * title = "Матриця:\n");
void print_max_cols(const int matr[M][N],
      const int m,
            const int n);
void change_matr(int matr[M][N],
      const int m,
            const int n);
// Головна функція з меню
int main() {
      setlocale(LC_ALL, "");
      int num; // Змінна для вибору пункту меню
      do {
            cout << "--------------------------------\n";
            cout << "1 - Matrix 24 (Максимуми по стовпцях)\n";
            cout << "2 - Matrix 53 (Заміна стовпців)\n";
            cout << "0 - Вихід\n";
            cout << "Ваш вибір -> ";
            cin >> num;
            // Виклик відповідної функції залежно від вибору
            switch(num) {
                  case 0:
                        cout << "Вихід...\n";
                        break;
                  case 1:
                        task1();
                        break;
                  case 2:
                        task2();
                        break;
                  default:
```

```
                        cout << "Невірний номер!\n";
            }
      } while(num != 0); // Цикл працює, поки не введено 0
      return 0;
}
// Функція введення розмірів та елементів матриці з перевіркою
void get_matr(int in_matr[M][N], int & in_m, int & in_n) {
      // Перевірка даних на коректність (від 2 до 50)
      do {
            cout << "Рядки (2-50): ";
            cin >> in_m;
            cout << "Стовпці (2-50): ";
            cin >> in_n;
      } while(in_n < 2 || in_n > N || in_m < 2 || in_m > M);
      cout << "Введіть " << in_m * in_n << " елементів:\n";
      // Цикл введення елементів
      for(int i = 0; i < in_m; i++) {
            for(int j = 0; j < in_n; j++) {
                  cin >> in_matr[i][j];
                  // Ігноруємо коми та пробіли, якщо користувач вводить в один
рядок
                  while(cin.peek() == ',' || cin.peek() == ' ') cin.ignore();
            }
      }
      // Очищення буфера вводу від зайвих символів
      while(cin.peek() != '\n' && cin.peek() != EOF) cin.ignore();
}
// Функція виведення матриці на екран
void show_matr(const int out_matr[M][N],
      const int m,
            const int n,
                  const char * title) {
      cout << title;
      // Прохід по всіх елементах та їх виведення через табуляцію
      for(int i = 0; i < m; i++) {
            for(int j = 0; j < n; j++) cout << out_matr[i][j] << "\t";
            cout << endl;
      }
}
// Завдання 1 (Matrix 24): Пошук максимального елемента у кожному стовпці
void print_max_cols(const int matr[M][N],
      const int m,
            const int n) {
      cout << "Максимуми по стовпцях:\n";
      // Зовнішній цикл по стовпцях (j)
      for(int j = 0; j < n; j++) {
            int mx = matr[0][j]; // Приймаємо перший елемент стовпця за максимум
            // Внутрішній цикл по рядках (i)
            for(int i = 1; i < m; i++)
                  if(matr[i][j] > mx) mx = matr[i][j]; // Оновлюємо максимум
```

```
            cout << mx << "\t"; // Виводимо знайдений максимум
      }
      cout << endl;
}
// Завдання 2 (Matrix 53): Перетворення матриці (обмін стовпців)
void change_matr(int matr[M][N],
      const int m,
            const int n) {
      // Шукаємо з кінця (справа наліво)
      for(int j = n - 1; j >= 0; j--) {
            bool pos = true; // Прапорець: чи всі елементи стовпця додатні
            for(int i = 0; i < m; i++) {
                  if(matr[i][j] <= 0) { // Якщо знайдено недодатний елемент
                        pos = false;
                        break; // Переходимо до наступного стовпця
                  }
            }
            // Якщо знайдено стовпець, де всі елементи додатні
            if(pos) {
                  // Міняємо місцями знайдений стовпець (j) і перший стовпець (0)
                  for(int i = 0; i < m; i++) {
                        int temp = matr[i][0];
                        matr[i][0] = matr[i][j];
                        matr[i][j] = temp;
                  }
                  break; // Робимо заміну тільки для одного (останнього) стовпця
і виходимо
            }
      }
}
void task1() {
      int matr[M][N], row, col;
      get_matr(matr, row, col); // Введення
      show_matr(matr, row, col); // Виведення початкової
      print_max_cols(matr, row, col); // Аналіз і виведення результату
}
void task2() {
      int matr[M][N], row, col;
      get_matr(matr, row, col); // Введення
      show_matr(matr, row, col, "Початкова матриця:\n"); // Виведення початкової
      change_matr(matr, row, col); // Перетворення
      show_matr(matr,  row,  col,  "Перероблена  матриця:\n");  //  Виведення
переробленої
}
```

ДОДАТОК Б

Скрін-шоти вікна виконання програми

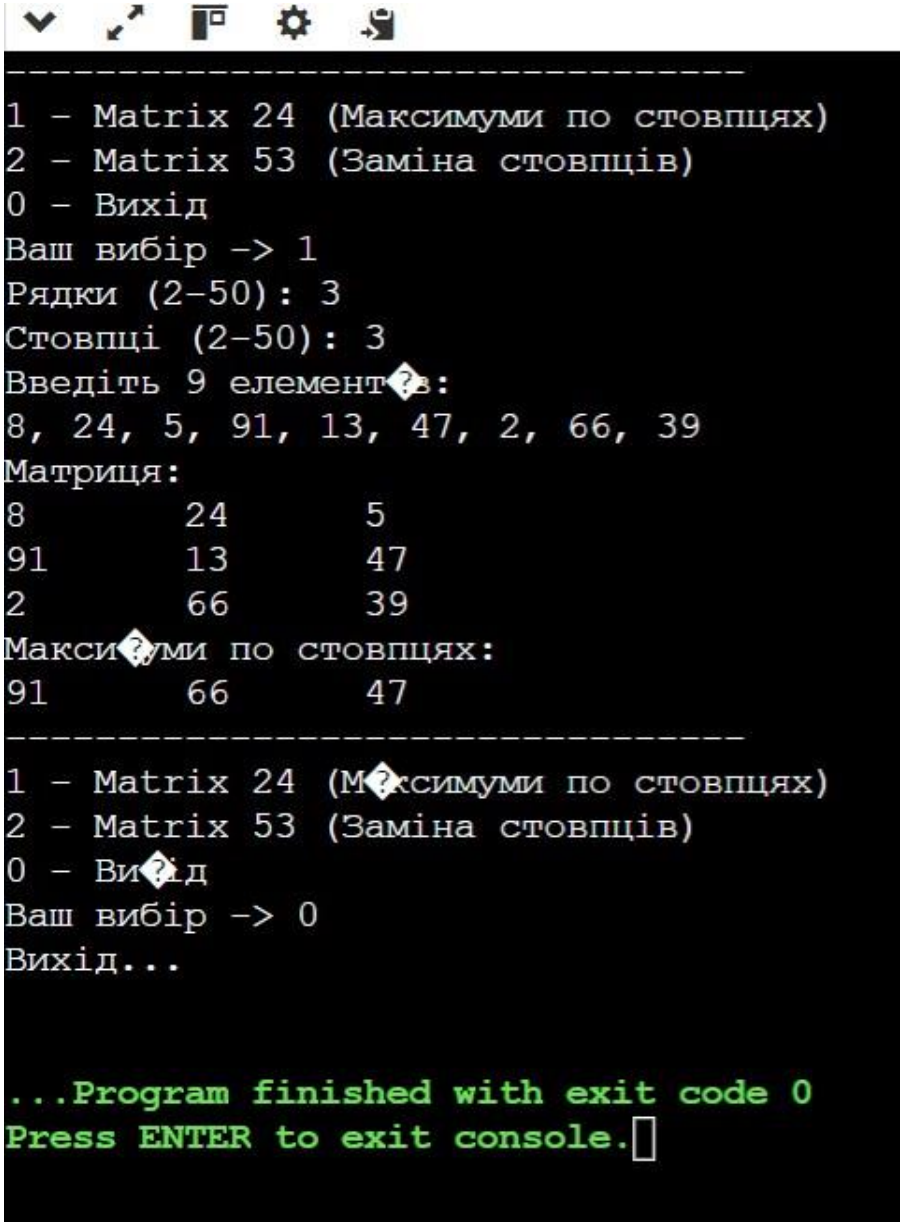


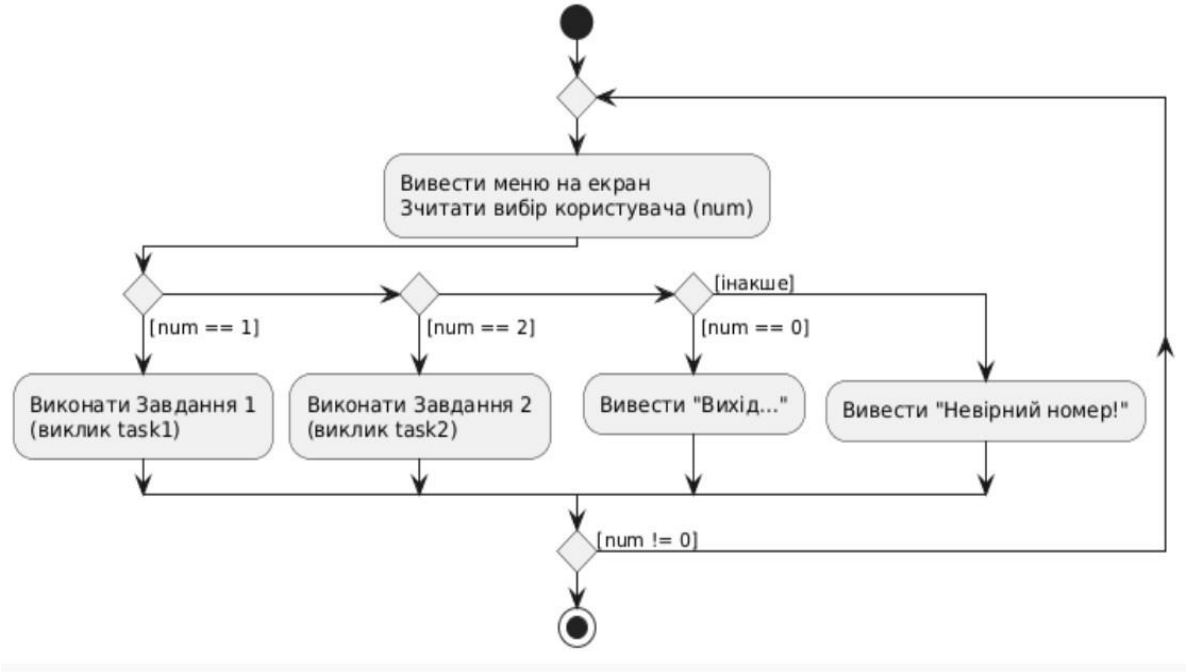
Рисунок Б.1 – Екран виконання програми для вирішення завдання Matrix 24

```
---------------------------------
1 - Matrix 24 (Максимуми по стовпцях)
2 - Matrix 53 (Заміна стовпців)
0 - Вихід
Ваш вибір -> 2
Рядки (2-50): 3
Стовпці (2-50): 3
Введіть 9 елемент?в:
52, 7, 19, 84, 3, 41, 95, 11, 6
Початкова матриц?:
52      7       19
84      3       41
95      11      6
Перероблена матриця:
19      7       52
41      3       84
6       11      95
---------------------------------
1 - Matrix 24 (Максимуми по стовпцях)
2 - Matrix 53 (Заміна стовпців)
0 - Вихід
Ваш вибір -> 0
Вихід...


...Program finished with exit code 0
Press ENTER to exit console.
```

Рисунок Б.2 – Екран виконання програми для вирішення завдання Matrix 53

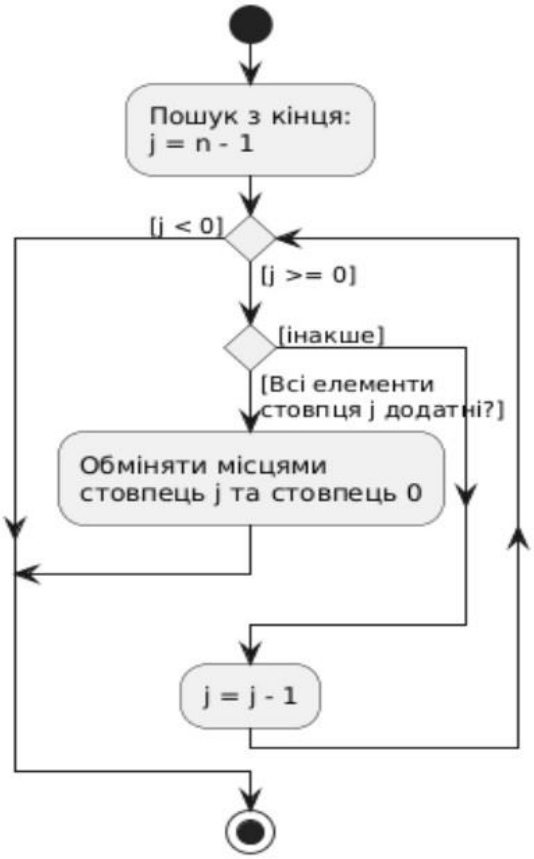
ДОДАТОК В
(UML діаграми завдань і усієї програми)



загальна UML-діаграма (головне меню)

UML-діаграма функції get_matr

UML-діаграма алгоритму Matrix 24



UML-діаграма алгоритму Matrix 53

ДОДАТОК Г

Діалог з ШІ для самоаналізу

Тестові питання:

1. Чому у функції get_matr параметри in_m та in_n передаються за посиланням (int & in_m), а не за значенням?

А) Щоб програма працювала швидше.

Б) Щоб зміни розмірів матриці, введені користувачем, збереглися у функції main.

В) Це вимога синтаксису C++ для двовимірних масивів.

Г) Щоб уникнути помилки переповнення пам'яті.

2. Що робить оператор break у функції change_matr (Завдання 53) після виконання блоку заміни стовпців?

А) Завершує роботу всієї програми.

Б) Переходить до наступної ітерації циклу по j.

В) Негайно перериває зовнішній цикл і завершує пошук, оскільки потрібний стовпець вже знайдено і замінено.

Г) Очищає пам'ять від змінної temp .

3. Яке ключове слово у функції show_matr гарантує, що функція випадково не змінить елементи статичної матриці під час її виведення на екран?

А) void

Б)return

В)static

г) const

4. Чому для перевірки введених розмірів матриці (від 2 до 50) використано цикл do-while , а не цикл for ?

А) Тому що for працює тільки з масивами.

Б) Тому що do-while гарантовано виконає запит хоча б один раз до перевірки умови.

В) do-while займає менше місця в пам'яті комп'ютера.

Г) Це випадковий вибір, обидва цикли працюють однаково.

5. Що відбудеться, якщо в Завданні 53 не знайдеться жодного стовпця, де всі елементидодатні?

А) Програма видасть помилку виходу за межі статичного масиву.
Б) Програма зациклиться назавжди.
В) Умова if (pos) ніколи не виконається, і матриця залишиться без змін.
Г) Програма замінить перший стовпець на нулі.

Відкриті питання:
1. Поясни, чому у Завданні 53 (заміна стовпців) логічніше шукати потрібний стовпець з кінця матриці (j = n - 1), а не з початку?
2. Яким чином у функції get_matr реалізовано захист від збою потоку введення cin, якщо користувач вводить числа через кому?
3. Що таке змінна-прапорець (flag), і як вона використовується у твоєму коді для Завдання 53?
4. Чому статичний двовимірний масив у C++ передається у функцію без знака амперсанда &, але при цьому функція (наприклад, change_matr) все одно може змінювати його оригінальні елементи?
5. Як працює алгоритм пошуку максимального елемента у функції print_max_cols ? З чим порівнюються наступні елементи?

Студент (Мої відповіді):
Тести: 1 - Б, 2 - В, 3 - Г, 4 - Б, 5 - В.
Відкриті: > 1. За умовою потрібно знайти останній такий стовпець. Якщо йти з кінця, то перший знайдений стовпець гарантовано буде останнім зліва направо. Це економить час, бо ми можемо одразу зробити break.

2. За допомогою функції перевірки наступного символу потоку. Якщо це кома або пробіл, ми викликаємо ігнорування, щоб пропустити його і не зламати зчитування чисел.

3. Змінна-прапорець у моєму коді - це змінна pos . Вона спочатку припускає, що всі числа додатні. Якщо трапляється число <= O, прапорець стає false . Потім по цьому прапорцю програма вирішує, чи робити заміну.
4. Тому що масив - це по суті і є вказівник. Він сам по собі працює з пам'яттю, тому амперсанд писати не треба, функція і так його бачить.
5. Береться перший елемент поточного стовпця і записується в змінну як еталон. Потім у циклі по рядках наступні елементи порівнюються з ним. Якщо

знаходиться більше число, еталон оновлюється.

ШІ (Викладач):
Аналіз тестових відповідей: Всі 5 відповідей абсолютно правильні. Оскільки це лише вибір варіантів, оцінка ймовірності використання ШІ не застосовується.
Оцінки: Т1: 5/5, Т2: 5/5, ТЗ: 5/5, Т4: 5/5, Т5: 5/5. (Сума: 25 балів).
Аналіз відкритих відповідей:
В1 (Пошук з кінця): Правильне і логічне пояснення. Мова природна. Ймовірність ШІ: 10%. Оцінка: 5/5.
В2 (Захист вводу): Точний опис. Ймовірність ШІ: 15%. Оцінка: 5/5.
В3 (Прапорець): Правильне пояснення патерну. Имовірність ШІ: 15%. Оцінка: 5/5.
В4 (Передача масиву): Відповідь частково правильна, але недостатньо розгорнута. Ти зрозумів ідею вказівника, але не пояснив механізм перетворення масиву на вказівник на його перший елемент при передачі у функцію. Ймовірність використання ШІ дуже низька (5%), відповідь виглядає написаною нашвидкуруч.
Штраф за ШІ не застосовується, але бали знято за неповноту розкриття теми.
Оцінка: 2/5.
В5 (Алгоритм максимуму): Чіткий алгоритмічний розбір простою мовою.
Ймовірність ШІ: 10%. Оцінка: 5/5.
Розрахунок загальної середньої оцінки:
Сума балів за тести: 25.
Сума балів за відкриті питання: 22 (5+5+5+2+5).
Загальна сума балів: 47 з 50 можливих.
Кількість питань: 10.
Середня оцінка: 47 / 10 = 4.7 балів.

Підсумок: Захист зараховано. Студент добре розуміє написані алгоритми та логіку програми. Рекомендую більш детально повторити теоретичну тему зв'язку статичних масивів та вказівників.